Lesson 6 — Cloud Management, Software Deployment, Kubernetes and Pods

INTE 22253 | Source: Lecture 06 Part I "Cloud Management & Software Deployment" (29 slides) and Lecture 06 Part II "Kubernetes Pods, From Scratch" (15 slides) | Builds on Lessons 2, 3 and 5

LEARNING OBJECTIVES (SLIDE 2)

- Explain the meaning of **cloud management** and **software deployment**.
- Describe how cloud resources are **provisioned** (including Infrastructure as Code).
- Understand the purpose and types of **load balancing**.
- Explain how **auto-scaling** improves application performance; horizontal vs vertical scaling.
- Identify the role of **Kubernetes**: architecture, objects, deployment, self-healing; and explain **Pods** with YAML and kubectl (Part II).
- Discuss ways to design cloud systems for **resilience, flexibility and cost-efficiency**.

1. Cloud management and software deployment (slides 3–5)

Cloud management is the process of **controlling and monitoring** cloud-based computing resources, storage, networks, applications, security policies, and users & access permissions. It ensures resources are **available, secure, efficiently used, properly monitored and cost-effective**.

Software deployment is the process of **making an application available for users**. Lifecycle:

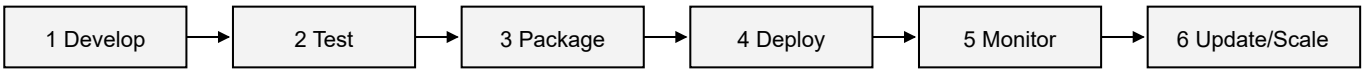


Figure 1 — Deployment lifecycle (slide 4). Cloud platforms simplify it with on-demand infrastructure and automated tools.

Service model (slide 5)	Description	Example
IaaS	Provides virtual machines, networks and storage	Amazon EC2
PaaS	Provides a platform for developing and deploying applications	Google App Engine
SaaS	Provides ready-to-use software over the Internet	Microsoft 365
CaaS Containers as a Service	Provides tools for running and managing containers	Amazon ECS

Cloud management applies across all of these models.

2. Resource provisioning and Infrastructure as Code (slides 6–7)

Resource provisioning = allocating computing resources to an application: virtual machines, containers, databases, storage, network connections and **security groups** (virtual firewalls that say which traffic may reach a resource).

Traditional provisioning	Cloud provisioning
Often manual · can be slow · may result in **over-provisioning** (buying more than needed) · difficult to reproduce consistently	Can be automated · uses templates and scripts · supports rapid deployment · can respond to changing demand

INFRASTRUCTURE AS CODE (IAC)

Managing infrastructure **through code instead of manual configuration**. Steps: **write configuration → review code → deploy infrastructure → monitor resources**.

Tools: Terraform, AWS CloudFormation, Azure Resource Manager, Ansible. **Benefits:** repeatable deployments, fewer configuration errors, faster infrastructure creation, easier version control, improved team collaboration.

Terraform example	Meaning
`resource "aws_instance" "web" {`	Declare an EC2 virtual machine named "web"
`  ami           = "ami-0abc123"`	Which machine image (OS template) to use
`  instance_type = "t3.micro"`	Size of the VM (vCPU/RAM)
`  count         = 2`	Create two identical instances
`}`	Run `terraform apply`: the tool creates exactly this; run it again and nothing changes (repeatable)

Because the file is in Git, every change is reviewed, versioned and can be rolled back — the same reasons code is kept in version control.

3. Load balancing (slides 8–9)

A **load balancer** distributes incoming network traffic among multiple servers or application instances.

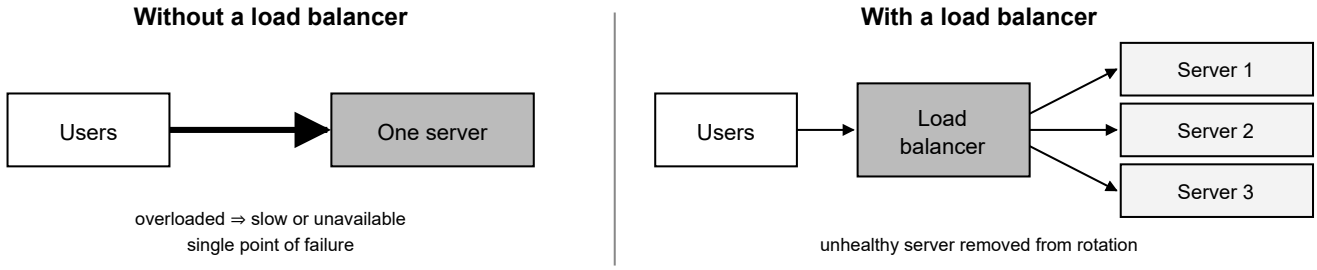


Figure 2 — Slide 8: without vs with a load balancer.

Benefits: improved performance · higher availability · better use of resources · reduced risk of failure · supports horizontal scaling. The LB also runs **health checks** and stops sending traffic to a failed server.

Type (slide 9)	Works at	Routes by / used for
Application load balancing	Application layer (Layer 7, HTTP)	URL paths, hostnames, HTTP headers, application content. E.g. `/images/*` to image servers, `/api/*` to API servers
Network load balancing	Network/transport layer (Layer 4, TCP/UDP)	IP address and port only; very fast. High-performance apps, TCP/UDP traffic, low latency
Global load balancing	Across regions (often via DNS)	Sends users to the nearest/healthy region: lower latency, survives regional failure, less pressure on one data centre

How it picks a server (standard algorithms): round robin (1, 2, 3, 1, 2, 3…), **least connections** (the server with fewest active connections), **weighted** (bigger servers get more). Example: 6 requests, round robin over 3 servers ⇒ S1 gets requests 1 and 4, S2 gets 2 and 5, S3 gets 3 and 6.

4. Auto-scaling (slides 10–12)

Auto-scaling automatically increases or decreases cloud resources according to demand. Slide 10: normal traffic ⇒ 2 instances; high traffic ⇒ 8; low traffic ⇒ back to 2.

Common scaling metrics: CPU utilization · memory usage · number of requests · queue length · response time · schedule / time of day.

Scaling type (slide 11)	Meaning	Example
Horizontal	Adding or removing instances	Servers 2 → 6
Vertical	Increasing the power of an existing server	More CPU or memory
Automatic	Resources change automatically based on rules	Add servers when CPU > 70%

Horizontal scaling is often preferred in the cloud because it improves availability and supports distributed applications.

Worked example: how many instances?

Each instance handles about 200 requests/s at 70% CPU. Rule: keep CPU near 70%; min 2, max 10. Traffic: 300 req/s at night, 1,500 req/s in a sale.

Traffic	Calculation	Instances
300 req/s	300 / 200 = 1.5 → round up = 2 (also the minimum)	2
1,500 req/s	1,500 / 200 = 7.5 → round up	8
3,000 req/s	3,000 / 200 = 15, but max is 10	10 (each runs hotter; raise the max or optimise)

Benefits (slide 12)

- Handles unexpected traffic
- Improves application availability
- Reduces manual work
- Avoids paying for unnecessary resources
- Supports business growth

Challenges

- Poorly configured rules may cause instability (scaling up and down repeatedly, "flapping")
- Scaling may take time (new instances need minutes to boot)
- Some applications are difficult to scale (they keep state on one server)
- Databases may become bottlenecks
- Sudden traffic increases can raise costs

Auto-scaling should be combined with monitoring, testing and cost controls.

5. Kubernetes (Lecture 06 slides 13–18; Part II slide 2)

5.1 The problem and the answer

The problem: your app is packaged into containers, and now there are dozens or thousands running across many machines. Who decides which machine runs which container? What happens when a machine dies? How do containers find each other?

The answer: Kubernetes is a **container orchestrator**, an open-source platform for **deploying, managing and scaling containerized applications**. You describe what you want ("run 3 copies of this app") and Kubernetes **places, monitors, restarts and scales** your containers across a **cluster** of machines automatically. It helps organisations: run containers across multiple machines · automatically restart failed containers · scale applications · perform software updates · manage networking and service discovery · **maintain the desired application state**.

Why containers? (slide 14) A container packages the application, its libraries and configuration. Advantages: consistent behaviour across environments, faster startup than VMs, efficient use of resources, easier deployment, simplified dependency management.

KEY IDEA: DESIRED STATE

Kubernetes is **declarative**: you declare the **desired state** (e.g. 3 replicas of web:v2). Controllers constantly compare it with the **actual state** and act to close the gap (start, stop or move Pods). This loop is why it "self-heals".

5.2 Kubernetes architecture (slide 15)

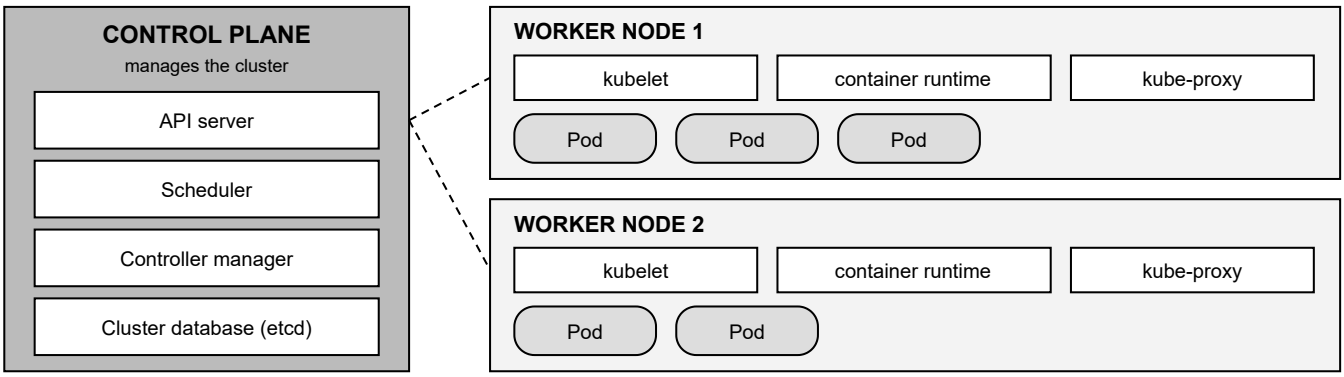


Figure 3 — A cluster = a control plane + one or more worker nodes (slide 15).

Component	Where	Job
API server	Control plane	Front door of the cluster: every command (kubectl, other components) goes through it
Scheduler	Control plane	Chooses which worker node each new Pod runs on, based on free resources
Controller manager	Control plane	Runs the controllers that keep actual state = desired state (e.g. "3 replicas")
Cluster database	Control plane	Stores the whole cluster state (in practice **etcd**, a key-value store)
kubelet	Each worker	Agent that starts/stops the node's Pods as told and reports their health
Container runtime	Each worker	Actually runs the containers (e.g. containerd, Docker)
kube-proxy	Each worker	Network rules so Services route traffic to the right Pods

5.3 Important Kubernetes objects (slide 16)

Object	Meaning
Pod	The **smallest deployable unit**; contains one or more containers
Deployment	Describes the **desired number and version** of application instances; recreates lost Pods, does rolling updates
Service	Provides **stable network access** to a group of Pods (Pods come and go and change IPs; the Service name/IP stays the same)
Ingress	Controls **external HTTP and HTTPS access** to applications (e.g. routes shop.com/api to the API Service)
ConfigMap & Secret	Store **configuration values** and **sensitive information** (passwords, keys) outside the image
Namespace	**Separates resources** within a cluster (e.g. dev, test, prod; or per team)

5.4 Deployment process (slide 17)

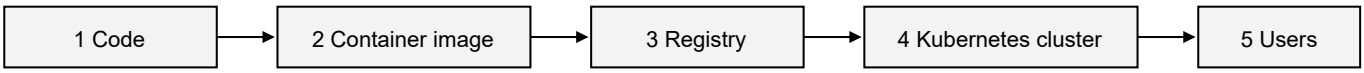


Figure 4 — From code to users (slide 17).

1. Build the application.
2. Create a container image.
3. Store the image in a container registry.
4. Write a Kubernetes deployment file (YAML).
5. Apply the configuration to the cluster (`kubectl apply`).
6. Kubernetes creates the required Pods.
7. A Service exposes the application.
8. Monitoring tools track performance and health.

deployment.yaml (worked example)	Meaning
`apiVersion: apps/v1` `kind: Deployment`	This document describes a Deployment
`metadata:` `  name: web`	Its name is "web"
`spec:` `  replicas: 3`	**Desired state: 3 Pods**
`  selector:` `  matchLabels: {app: web}`	Manage the Pods labelled app=web

`  template:`
`  metadata:`
`  labels: {app: web}`
	The Pod template: every Pod gets label app=web

`spec:`
`  containers:`
`  - name: web`
`  image: myrepo/web:v2`
	Each Pod runs one container from image web:v2 (pulled from the registry)

If a node dies and one Pod is lost, the Deployment controller sees 2 ≠ 3 and the scheduler starts a new Pod on another node. Changing `image` to v3 triggers a rolling update.

5.5 Self-healing and updates (slide 18)

Self-healing

- Restarting failed containers
- Replacing unhealthy Pods
- Rescheduling workloads on available nodes
- Maintaining the desired number of replicas

Also supports

- **Rolling updates**: replace Pods a few at a time with the new version, so the app never goes fully down
- **Rollbacks**: return to the previous version
- **Health checks** (probes): test whether a container is alive/ready
- Automated **service discovery**

Rolling update trace (3 replicas, v1 → v2): start v1 v1 v1 → add one v2 → when healthy remove one v1 (v1 v1 v2) → v1 v2 v2 → v2 v2 v2. At every step some Pods serve users: zero downtime. If v2 fails its health checks, `kubectl rollout undo` returns to v1.

6. Pods from scratch (Lecture 06 Part II)

6.1 The apartment analogy (slide 3)

Apartment	Kubernetes Pod
Roommates share one street address and one set of utilities (water, electricity); they walk between rooms freely; if the building is demolished all roommates are displaced together.	Containers in a Pod share one network address (IP) and, optionally, storage volumes; they are always scheduled onto the same node; if the Pod is removed every container in it goes with it.
apartment = Pod · address = Pod IP · utilities = shared volumes · roommates = containers. A Pod is **not** a container: it is a thin wrapper that holds one or more containers plus the resources they share.	

6.2 What is a Pod, formally? (slides 4–5)

The **smallest deployable unit** in Kubernetes. Kubernetes never schedules a bare container; it always schedules a Pod.

1. **Shared context**: containers share the same network namespace and, optionally, storage volumes.
2. **Co-scheduled**: all containers in a Pod are always placed on the same node and scheduled together.
3. **Single IP address**: the Pod gets one cluster IP; containers inside talk to each other over **localhost**.
4. **Ephemeral by design**: Pods are disposable — Kubernetes **replaces** failed Pods rather than repairing them in place.

Without Pods	With Pods
No natural way to say "these two containers must always run together"	The Pod spec explicitly groups containers that belong together
No shared network identity for tightly-coupled helpers (logging, proxying)	Every container shares one IP and talks over localhost
No shared storage between cooperating containers without extra plumbing	A volume declared once in the Pod spec is available to every container

6.3 Anatomy of a Pod (slide 6)

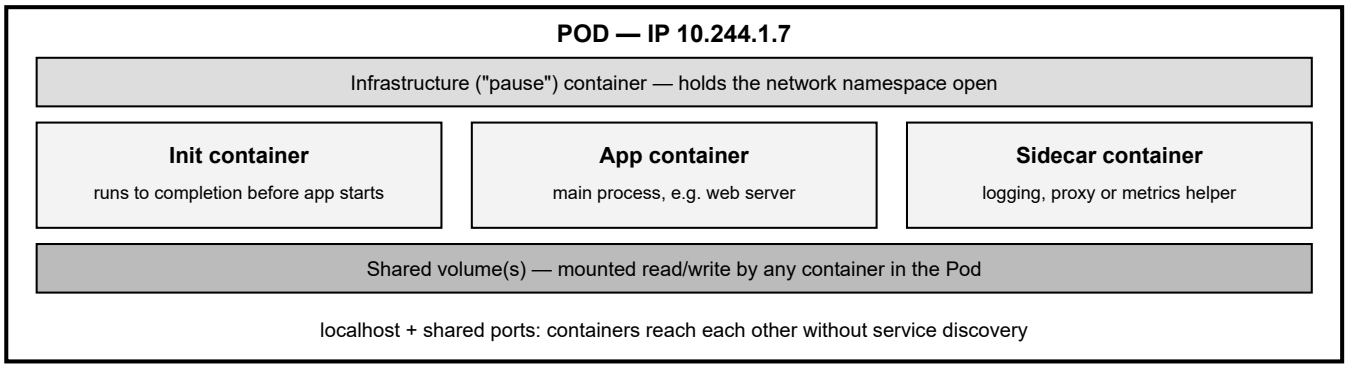


Figure 5 — Every Pod is one network and storage boundary, however many containers it holds (slide 6).

Key facts: one IP per Pod, shared by all containers · containers communicate via localhost · volumes outlive container restarts · init containers run in order, then exit · defined in a single YAML manifest.

6.4 Your first Pod: YAML and kubectl (slides 7–8)

```
apiVersion: v1
kind: Pod
metadata:
  name: my-first-pod
spec:
  containers:
  - name: web
    image: nginx:latest
    ports:
    - containerPort: 80
```

Field	Meaning
apiVersion / kind	This document describes a Pod
metadata.name	The Pod's name: how you refer to it later
spec.containers	The list of containers to run in this Pod
image	Which container image to pull and run
containerPort	The port the container listens on

YAML is declarative: you describe the desired state; Kubernetes makes it real.

```
$ kubectl apply -f pod.yaml          # send the YAML; Kubernetes creates the Pod to match
pod/my-first-pod created
$ kubectl get pods                   # list Pods and their status at a glance
NAME          READY   STATUS    RESTARTS   AGE
my-first-pod  1/1     Running   0          8s
$ kubectl describe pod my-first-pod  # full detail: events, container status, IP, node
```

Reading the output: READY 1/1 = 1 of 1 containers ready; STATUS Running = phase; RESTARTS 0 = never restarted; AGE 8s.

6.5 Examples: localhost, shared volume, sidecar, init container (slides 9–12)

Pattern	What the slide's YAML does	Why
Talking over localhost (slide 9)	Containers `app` (port 8080) and `metrics-sidecar` in one Pod; the sidecar calls `curl localhost:8080/metrics`	Same Pod ⇒ same localhost; no networking setup required
Sharing files (slide 10)	Containers `writer` and `reader` both mount volume `shared-data` (type **emptyDir**) at `/data`	emptyDir is created fresh when the Pod starts; a file the writer creates the reader sees immediately; survives a **container** restart, deleted when the **Pod** is removed
Sidecar pattern (slide 11)	`app` writes logs to `/var/log/app`; `log-shipper` (fluentd) reads the same `logs` volume and ships them	App team keeps simple code ("write logs to a file"); platform team owns and upgrades the shipper without touching the app; both still restart, scale and schedule together
Init container (slide 12)	`wait-for-db` runs `until nc -z db 5432; do sleep 2; done` before `app`	Retries the DB connection every 2 s; `app` does not start until it exits successfully. Multiple init containers run one at a time, in order

```
spec:
  initContainers:
  - name: wait-for-db
    image: busybox
    # nc -z = only test whether port 5432 on host "db" is open
    command: ["sh", "-c", "until nc -z db 5432; do sleep 2; done"]
  containers:
  - name: app
    image: my-app:v1
```

6.6 Pod lifecycle (slide 13)

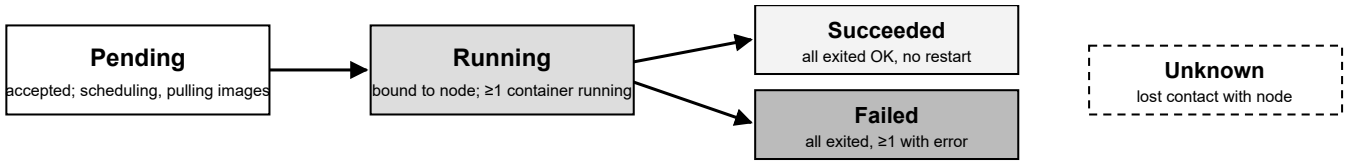


Figure 6 — Pod phases (slide 13).

```
$ kubectl get pod my-first-pod --watch
NAME           READY   STATUS              RESTARTS   AGE
my-first-pod   0/1     Pending             0          1s
my-first-pod   0/1     ContainerCreating   0          2s
my-first-pod   1/1     Running             0          4s
```

6.7 Restart policy and self-healing (slide 14)

restartPolicy	Behaviour	Used for
Always (default)	Restart no matter how the container exits	Web servers, APIs
OnFailure	Restart only on a non-zero exit code (an error)	Batch jobs
Never	Never restart, success or failure	One-shot tasks

What "self-healing" actually means: a crashed container restarts **in place, in the same Pod**, following the policy, with a growing **backoff** delay between attempts (status **CrashLoopBackOff**: it keeps crashing, so Kubernetes waits longer each time: about 10 s, 20 s, 40 s… up to 5 min). But if the **Pod itself is deleted or its node fails, nothing recreates it** — that is the job of a **controller such as a Deployment**. Durability lives one layer up.

Worked example: a batch Pod with `restartPolicy: OnFailure` exits with code 1 ⇒ restarted. Next run exits with code 0 ⇒ not restarted; phase **Succeeded**. With `Never`, the first exit code 1 ⇒ phase **Failed**. With `Always` a web server that exits even with 0 is restarted.

Key takeaways (Part II slide 15): What — a Pod bundles containers behind one IP and optional shared volumes. **Why** — "these containers belong together, always". **How** — YAML (apiVersion, kind, metadata, spec) + kubectl. **Behaviour** — Pending → Running → Succeeded/Failed; containers restart per policy. **Beyond Pods** — Deployments recreate lost Pods.

7. Designing for resilience, flexibility and cost-efficiency (slides 19–21)

Resilience (slide 19)	Flexibility (slide 20)	Cost-efficiency (slide 21)
Ability to keep operating when failures occur. • Use multiple servers or AZs • Avoid single points of failure • Use load balancers • Replicate important data • Automate recovery • Use health checks • Back up data regularly • Test DR plans "A resilient system expects that failures will occur — and prepares for them."	*Adapts to changing business and technical requirements.* • Modular application components • Separate services where appropriate • Containers for portability • Automate infrastructure and deployments • Avoid unnecessary dependence on one provider • APIs and standard interfaces • Support multiple environments (dev, test, prod)	*Cloud costs can increase quickly without management.* • Auto-scaling; shut down unused resources • Correct instance size • Reserved/discounted pricing where suitable • Monitor usage; set budgets and alerts • Appropriate storage tiers • Remove unused disks, IPs, snapshots • Serverless for suitable workloads Goal: best **value** for the required performance and reliability, not simply the lowest cost.

8. Monitoring, security, example, best practices (slides 22–29)

Monitoring and observability (slide 22). Important areas: CPU and memory, response time, error rates, network traffic, active users, deployment success/failure, cloud spending. Observability uses **metrics** (numbers over time), **logs** (event records), **traces** (the path of one request through many services), **alerts** and **dashboards**. Goal: find problems before users are seriously affected.

Security in cloud deployment (slide 23): strong IAM · principle of **least privilege** · encrypt data in transit and at rest · secure container images · update software regularly · store secrets securely · network segmentation · monitor suspicious activity · vulnerability scanning. Security is a **shared responsibility** between provider and customer.

Example: online shopping application (slide 24) uses containers for app services, Kubernetes to manage them, a load balancer for customer requests, auto-scaling during sales, a managed database for orders, and object storage for product images. During a major sale: more customers → more application Pods → load balancer distributes traffic → application remains available.

Advantages of cloud deployment (slide 25)

- Faster time to market
- Flexible resource usage
- Global availability
- Automated scaling
- Easier disaster recovery
- Reduced physical infrastructure
- Support for CI/CD

Challenges of cloud management (slide 26)

- Unexpected costs; security misconfigurations
- Vendor dependence; complex Kubernetes management
- Network failures; data privacy concerns
- Difficult troubleshooting; lack of skilled personnel
- Poorly designed applications

Best practices (slide 27): automate infrastructure and deployment · load balance critical apps · configure auto-scaling carefully · deploy across multiple failure zones · monitor performance and cost · secure identities, networks and data · test backup and recovery · use containers consistently · keep configurations under version control · review architecture regularly. Cloud technology "does not automatically guarantee a reliable or efficient system".

9. Exam-style questions with model answers

Q1. Define cloud management and software deployment. (4 marks)
Cloud management is the process of controlling and monitoring cloud computing resources, storage, networks, applications, security policies and user access so that they are available, secure, efficiently used, monitored and cost-effective. Software deployment is the process of making an application available to users: develop, test, package, deploy, monitor, update/scale.

Q2. What is Infrastructure as Code? Give two tools and four benefits. (6)
IaC manages infrastructure through code files instead of manual configuration (write, review, deploy, monitor). Tools: Terraform, AWS CloudFormation, Azure Resource Manager, Ansible. Benefits: repeatable deployments, fewer configuration errors, faster creation, version control, better team collaboration.

Q3. What might happen if an application has no load balancer? Describe three types of load balancing. (7)
All traffic hits one server; under load it becomes slow or unavailable, and it is a single point of failure; horizontal scaling is impossible. Application LB: layer 7, routes by URL path, hostname, headers, content. Network LB: layer 4, routes TCP/UDP traffic by IP and port, high performance, low latency. Global LB: distributes users across geographic regions to cut latency and survive regional failures.

Q4. Why is auto-scaling useful during unexpected traffic increases? State two challenges. (5)
It adds instances automatically when metrics such as CPU, requests or queue length rise, so the application stays available and responsive without manual action, and removes them afterwards so you do not pay for idle capacity. Challenges: badly configured rules cause instability; scaling takes time; some apps or databases are hard to scale; sudden spikes raise costs.

Q5. Distinguish horizontal and vertical scaling. (3)
Horizontal: add or remove instances (2 → 6 servers); improves availability, preferred in the cloud. Vertical: make one server more powerful (more CPU/RAM); limited by machine size.

Q6. What problem does Kubernetes solve? Describe its architecture. (8)
With many containers across many machines, someone must decide placement, restart failures, scale, update and connect them. Kubernetes is a container orchestrator that does this automatically to maintain a declared desired state. A cluster has a control plane (API server: entry point; scheduler: places Pods on nodes; controller manager: keeps actual state equal to desired; cluster database: stores state) and worker nodes (kubelet: runs and reports Pods; container runtime: runs containers; kube-proxy: networking for Services) that host Pods.

Q7. Explain Pod, Deployment, Service, Ingress, ConfigMap/Secret and Namespace. (6)
Pod: smallest deployable unit with one or more containers. Deployment: desired number and version of instances. Service: stable network access to a group of Pods. Ingress: external HTTP/HTTPS access. ConfigMap/Secret: configuration values and sensitive data. Namespace: separates resources within a cluster.

Q8. What is a Pod? Why does Kubernetes schedule Pods rather than containers? (6)
A Pod is a thin wrapper around one or more containers that share one network namespace (one IP, talk over localhost) and optional storage volumes, are always co-scheduled on the same node, and are ephemeral. Kubernetes uses Pods because they explicitly group containers that must run together, give tightly coupled helpers a shared network identity, and let cooperating containers share volumes without extra plumbing.

Q9. Explain the sidecar pattern and the init container with examples. (6)
Sidecar: a helper container in the same Pod as the app, e.g. the app writes logs to a shared emptyDir volume and a fluentd log-shipper reads and forwards them; teams can change each independently while they still scale and schedule together. Init container: runs to completion before app containers start, e.g. wait-for-db loops `nc -z db 5432` every 2 s and the app starts only after the database is reachable; multiple init containers run in order.

Q10. Describe the Pod lifecycle phases and the three restart policies. Does restarting a container recreate a deleted Pod? (8)
Pending (accepted, waiting for scheduling/images), Running (bound to a node, at least one container running), Succeeded (all exited successfully, no restart), Failed (all exited, at least one with error), Unknown (state cannot be confirmed, lost contact with node). Policies: Always (default, web servers), OnFailure (restart on non-zero exit, batch jobs), Never (one-shot). Restarts happen in place with growing backoff (CrashLoopBackOff). No: if the Pod is deleted or its node fails nothing recreates it; a controller such as a Deployment must do that.

Q11. How can an organisation reduce unnecessary cloud costs? Why design for failure? (6)
Auto-scaling, shutting down unused resources, right-sizing instances, reserved/discounted pricing, monitoring usage with budgets and alerts, storage tiers, removing unused disks/IPs/snapshots, serverless for suitable workloads. Design for failure because in large systems failures will occur; using multiple AZs, no SPOFs, load balancers, replication, automated recovery, health checks, backups and tested DR plans keeps the service running.

10. Common mistakes

Mistake	Correct idea
A Pod is a container	A Pod wraps one or more containers that share an IP and volumes.
Containers in a Pod talk through a Service	They use localhost; Services are for reaching Pods from elsewhere.
Restart policy brings back a deleted Pod	It only restarts containers in place; a Deployment recreates Pods.
emptyDir data survives Pod deletion	It survives container restarts but is deleted with the Pod.
Scheduler runs containers on nodes	The scheduler chooses the node; the kubelet and container runtime run them.
Network LB routes by URL	Application LB (layer 7) routes by URL/headers; network LB (layer 4) by IP/port.
Vertical scaling is preferred in the cloud	Horizontal is preferred: availability and distribution.
IaC = writing application code	IaC = infrastructure defined in code files (Terraform, CloudFormation).
Cost-efficiency = lowest possible cost	Best value for the required performance and reliability.

Cheat sheet

Management & deployment	
Cloud mgmt	Control + monitor compute, storage, networks, apps, security policies, users/access
Deployment	Develop → test → package → deploy → monitor → update/scale
CaaS	Containers as a Service (Amazon ECS)
Provisioning	Allocate VMs, containers, DBs, storage, network, security groups; cloud = automated, templates
IaC	Write → review → deploy → monitor. Terraform, CloudFormation, ARM, Ansible. Repeatable, fewer errors, faster, versioned, collaborative
LB types	Application (L7: URL, host, headers) · Network (L4: TCP/UDP, fast) · Global (regions)
Auto-scaling	Metrics: CPU, memory, requests, queue length, response time, schedule. 2 → 8 → 2
Scaling	Horizontal (instances) · vertical (power) · automatic (rules)
Design	Resilience (expect failure) · flexibility (modular, portable, no lock-in) · cost (best value)
Observability	Metrics, logs, traces, alerts, dashboards

Kubernetes	
What	Container orchestrator; declarative desired state
Control plane	API server, scheduler, controller manager, cluster DB (etcd)
Worker	kubelet, container runtime, kube-proxy, Pods
Objects	Pod, Deployment, Service, Ingress, ConfigMap/Secret, Namespace
Deploy	Code → image → registry → YAML → kubectl apply → Pods → Service → monitor
Self-heal	Restart, replace, reschedule, keep replicas; rolling updates, rollbacks, health checks

Pods	
Pod	Smallest unit; shared context; co-scheduled; one IP (localhost); ephemeral
Inside	Pause container, init, app, sidecar, shared volumes
YAML	apiVersion, kind, metadata, spec
kubectl	apply -f · get pods · describe pod · --watch
emptyDir	New at Pod start; survives container restart; deleted with Pod
Phases	Pending → Running → Succeeded / Failed; Unknown
Restart	Always (default, web) · OnFailure (batch) · Never (one-shot); CrashLoopBackOff
Durability	Deployment recreates lost Pods